

A Comparative Study of Deep RL Algorithms

Andrea Tarabay, Joelle Alachkar, Omar Shibli



Motivation and Objective

- Deep RL algorithms use neural networks to learn decision-making policies through interaction with an environment.
- However, they differ in how they **optimize policies**, **reuse data**, **stabilize training**, and **handle different action spaces**.
- We implement **DQN**, **PPO**, **SAC**, and **TD3** from **scratch** and evaluate them on Gym control tasks.
- The comparison focuses on stability, sample efficiency, hyperparameter sensitivity, and action-space suitability. **The goal is to identify which design choices matter most in practice.**

Algorithms at a Glance

Alg.	What is learned?	How does it work?	Stabilization
DQN	Learns an action-value function $Q_\theta(s, a)$ for discrete actions. The policy is implicit: $\pi(s) = \arg \max_a Q_\theta(s, a)$.	Uses Bellman targets to regress the Q-network: $y = r + \gamma \max_{a'} Q(s', a')$ and minimizes the temporal-difference error.	Uses replay buffer, ϵ -greedy exploration, and fixed targets.
PPO	Learns a stochastic actor $\pi_\theta(a s)$ and a critic $V_\psi(s)$. The actor is categorical for discrete tasks and Gaussian for continuous tasks.	Updates the policy using the clipped probability-ratio objective: $r_t(\theta) = \frac{\pi_\theta(a_t s_t)}{\pi_{\theta_{\text{old}}}(a_t s_t)}$ to avoid overly large policy changes.	Uses GAE, advantage normalization, gradient clipping, and multiple epochs per rollout.
SAC	Learns a stochastic Gaussian actor, two Q-functions Q_1, Q_2 , and a soft value estimate.	Optimizes reward together with entropy: $\mathbb{E}[r(s, a) + \alpha \mathcal{H}(\pi)]$ so the policy remains exploratory.	Uses entropy regularization, replay buffer, twin Q-functions, tanh correction, and Polyak target updates.
TD3	Learns a deterministic actor $\pi_\theta(s)$ and two critics $Q_{\theta_1}(s, a)$, $Q_{\theta_2}(s, a)$.	Uses clipped double Q-learning: $y = r + \gamma \min_{i=1,2} Q_{\theta_i}(s', \tilde{a})$ where $\tilde{a}$ is a smoothed target action.	Uses twin critics, delayed actor updates, target policy smoothing, and Polyak averaging.

Systematic comparison. DQN is value-based and restricted to discrete actions. PPO directly optimizes a policy and is stable but on-policy. SAC and TD3 are off-policy actor-critic methods designed for sample-efficient continuous control.

Experimental Setup

We compare four deep RL algorithms on low-dimensional Gym control tasks with both **discrete** and **continuous** action spaces.

Benchmark environments.

Environment	Action Space	Control Task	Tested Algorithms
CartPole-v1	Discrete	Pole balancing	DQN, PPO
Acrobot-v1	Discrete	Swing-up	DQN, PPO
Pendulum-v1	Continuous	Torque control	PPO, SAC, TD3
MountainCarContinuous-v0	Continuous	Goal reaching	SAC, TD3

Evaluation protocol.

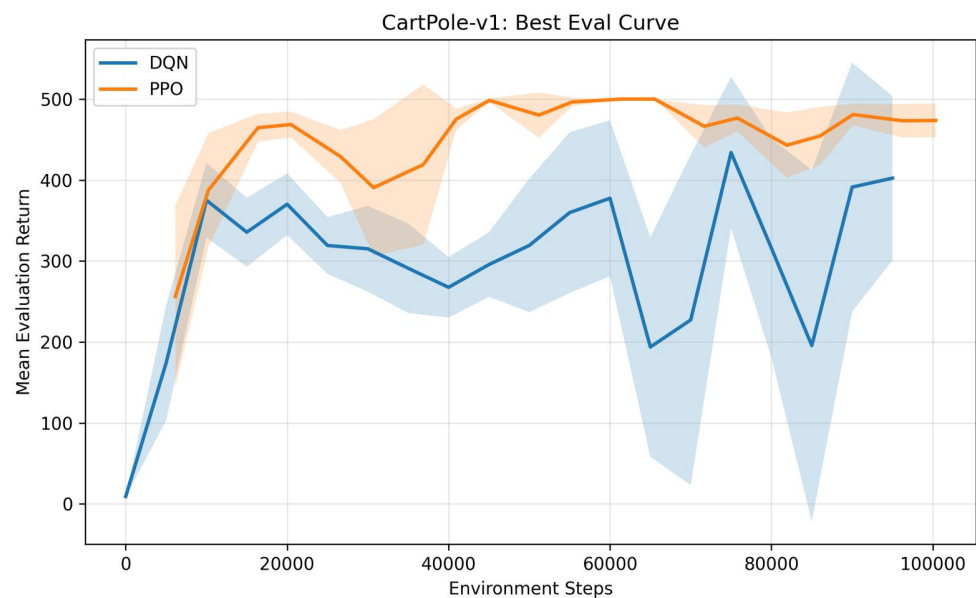
Metric	Mean episodic return with standard-deviation bands
Random seeds	3
Training budget	100,000 steps for DQN, PPO, TD3 30,000 steps for SAC due to faster convergence

Results

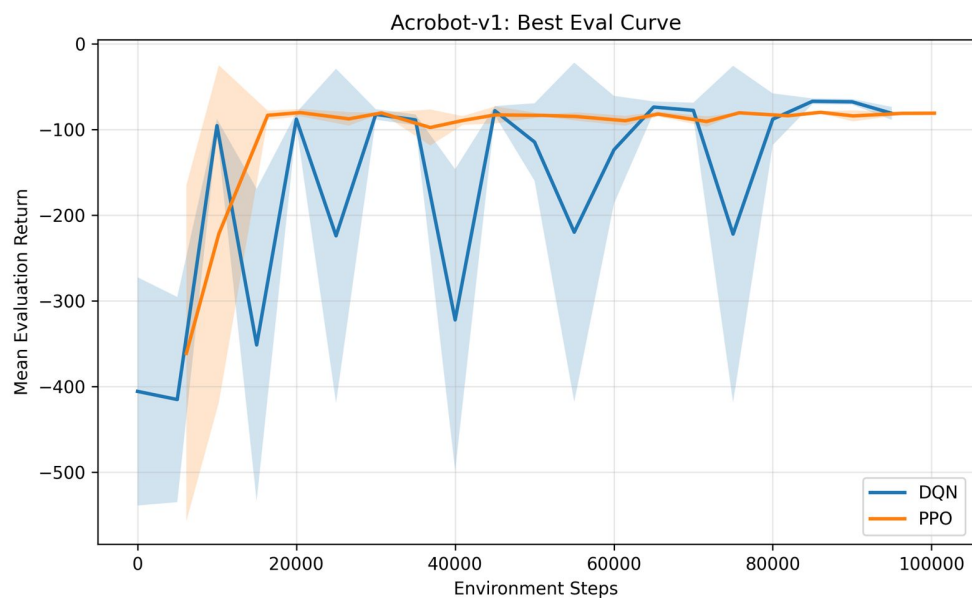
Discrete Action-Space Results

CartPole-v1: Both algorithms learn the task **quickly**, but PPO maintains **near-optimal returns** more consistently, while DQN shows larger **oscillations** later in training.

Acrobot-v1: PPO reaches a **stable swing-up policy** and keeps performance steady, whereas DQN improves but suffers from **repeated drops**.



CartPole-v1



Acrobot-v1

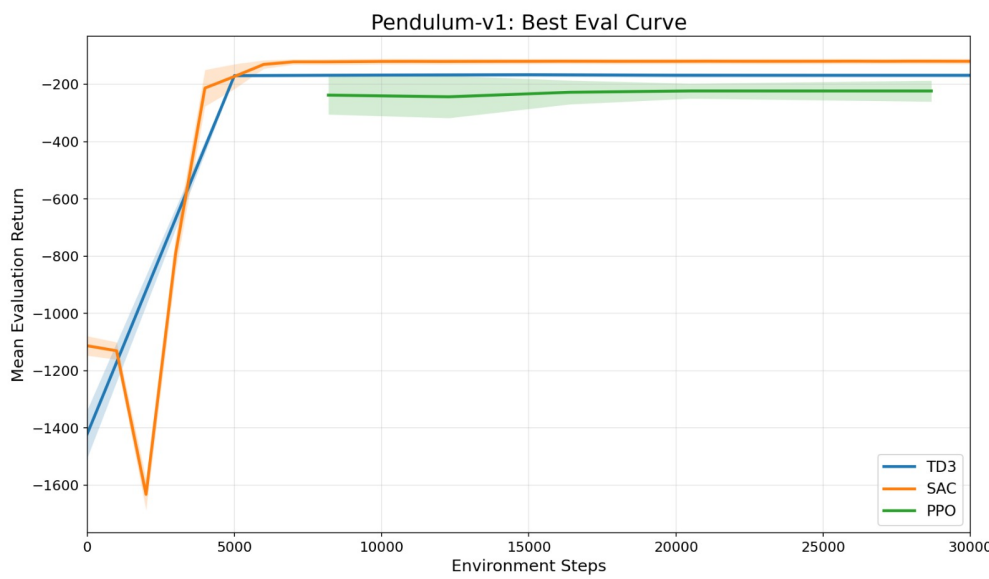
Continuous Action-Space Results

Pendulum-v1: PPO shows **limited progress**, while SAC improves fastest, reaching around ~ -120 . TD3 also learns successfully, reaching around -170 and remaining stable.

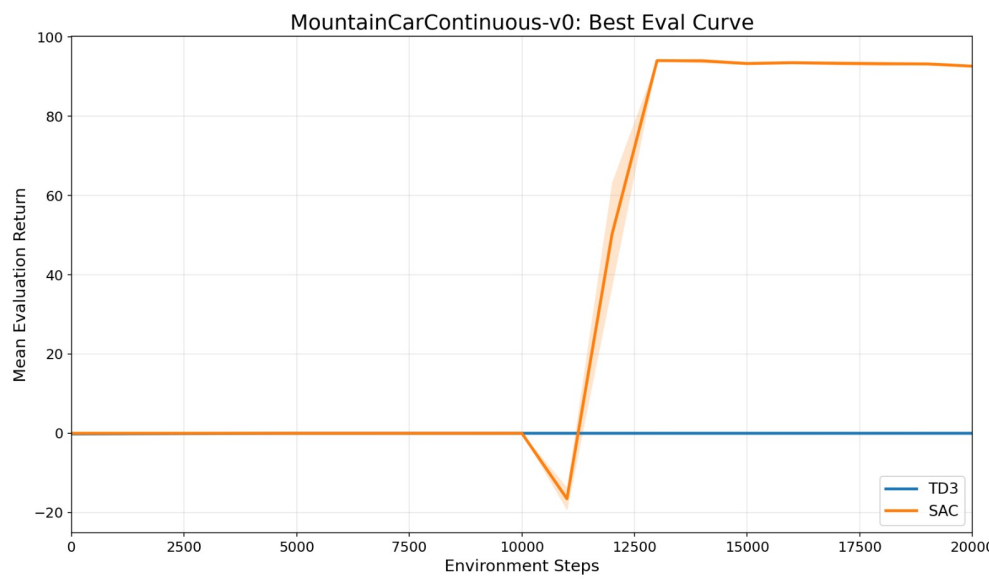
MountainCarContinuous-v0: Both SAC and TD3 failed to discover the **goal-reaching behavior**. SAC often learned an **inaction policy**, while TD3 stayed close to zero return even after 500k steps.

Overall: In our experiments, PPO performs best on the two discrete tasks, while SAC and TD3 are more suitable for sample-efficient continuous control.

Results



Pendulum-v1



MountainCarContinuous-v0

Off-policy actor-critic methods perform best on Pendulum-v1. Without reward shaping, both SAC and TD3 struggled to discover the goal-reaching behavior and often converged to low-action policies. With reward shaping, denser progress feedback helped the agent escape this local optimum and reach the goal more reliably.

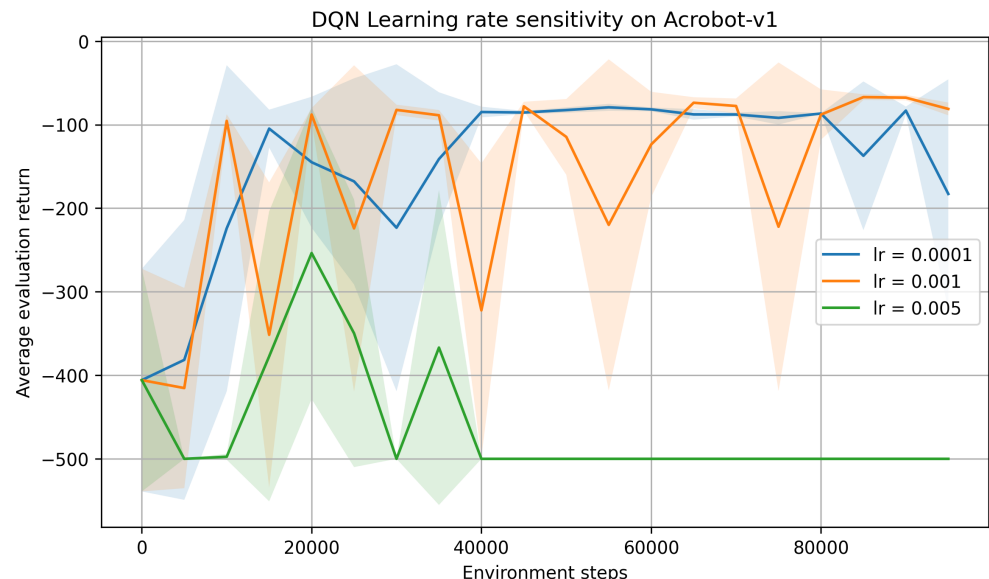
Hyperparameter Sensitivity

Discrete

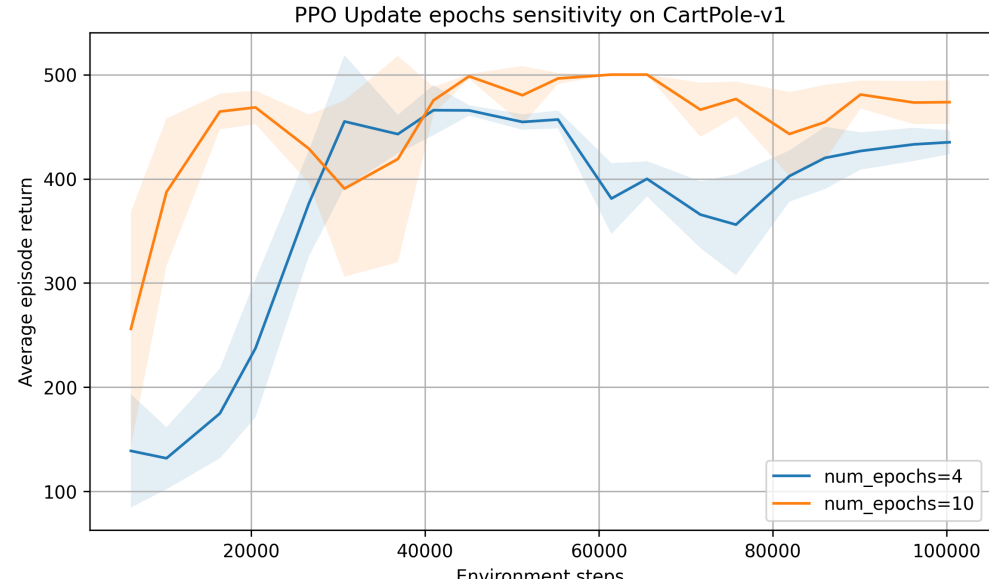
Alg.	Hyperparameter	Effect when increased
DQN	Learning rate	Faster learning, but too high caused unstable Q-updates and drops.
	Discount factor γ	Better delayed-reward learning, especially on Acrobot.
	ϵ -decay length	More exploration for longer, giving smoother learning.
	Replay buffer size	Helped Acrobot stability, but not always CartPole.
	Network size	Small effect on low-dimensional tasks.
PPO	Clip range ϵ	Larger updates, but more fluctuations.
	Learning rate	Faster early learning, but lower stability.
	Update epochs	Better sample reuse and faster learning.
	Rollout actors	More actors did not always help with fixed batch size.

Continuous

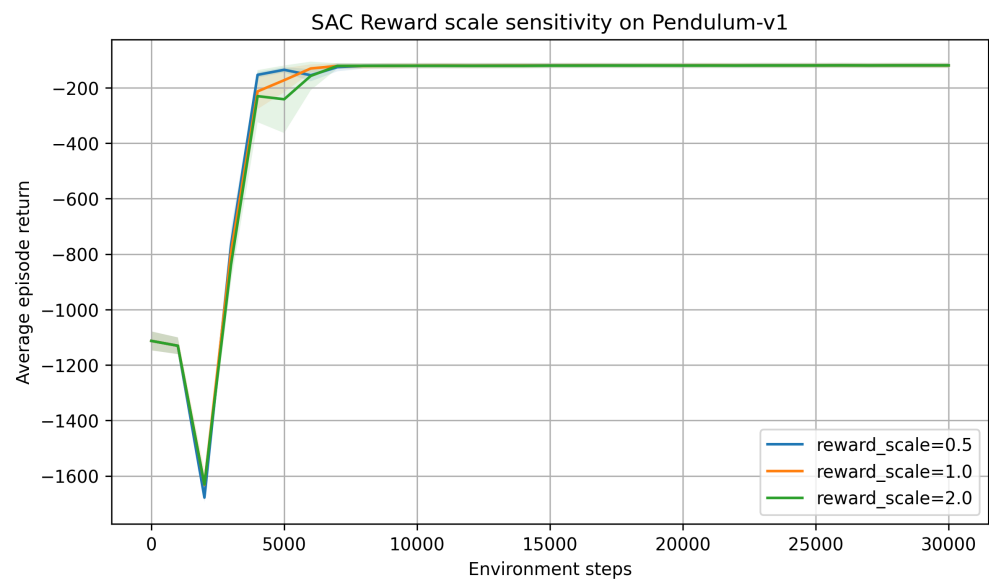
Alg.	Hyperparameter	Effect when increased
SAC	Entropy coefficient α	Small final effect; SAC remained robust.
	Reward scale	Slight final improvement, but differences were small.
	Target update τ	No major performance change observed.
TD3	Policy delay d	Slower early actor learning, similar final performance.
	Target policy noise	Little final effect; all settings converged similarly.
	Exploration noise	Better exploration, but more early variability.
	Batch size / τ	Default setting already gave stable convergence.



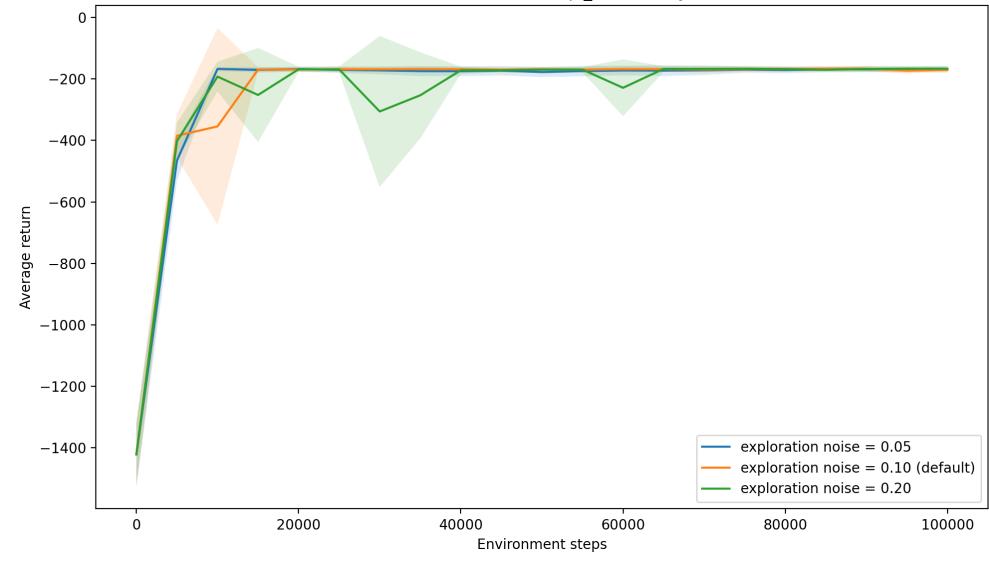
DQN: Higher LR speeds learning, but can destabilize Q-updates.



PPO: More epochs improve sample reuse and early learning.



SAC: Reward scale has only a small final effect.



TD3: Higher exploration noise improves exploration but adds variability.

Key Takeaways

- PPO was the most stable method for discrete-control tasks.
- DQN learned successfully, but was more sensitive to hyperparameters and Q-value instability.
- SAC and TD3 were more suitable for smooth continuous-control tasks such as Pendulum, but sparse-reward exploration remained difficult on MountainCarContinuous.

Main conclusion: Algorithm design choices strongly affect performance: replay buffers improve sample efficiency, clipping stabilizes policy updates, entropy improves exploration, and twin critics reduce over-estimation.

References

[1] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*, pages 1587–1596, 2018.

[2] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pages 1861–1870, 2018.

[3] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fiedjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[4] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.